BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên: Trương Quốc Lê Huy	
MSSV / Lớp: 3123410132	

Mục tiêu học tập

• Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).

• Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.

• Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.

• Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.

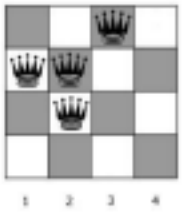
• Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Giải:

Bước 1: Xác định trạng thái ban đầu..

Ta đếm được 4 cặp quân hậu đang xung đột:

A3 – B2	Cùng đường chéo
B1 – B2	Cùng hàng B
B1 – C2	Cùng đường chéo
B2 – C2	Cùng cột 2

Vì vậy, trạng thái ban đầu có 4 xung đột.

Bước 2: Tìm trạng thái láng giềng có ít xung đột nhất.

Theo giả mã, ta di chuyển một quân hậu đến một ô trống bằng nước đi hợp lệ của quân hậu, sau đó chọn trạng thái có số xung đột thấp nhất.

Một nước đi giúp giảm số xung đột là di chuyển quân hậu ở ô B2 đến D4. Đường đi qua C3 đang trống nên nước đi hợp lệ.

Trạng thái mới:

$$S_1 = \{A3, B1, C2, D4\}$$

Sau khi di chuyển, chỉ còn cặp B1 và C2 xung đột trên đường chéo. Số xung đột giảm từ 4 xuống 1.

Kết quả: Sau một vòng lặp, bốn quân hậu nằm tại A3, B1, C2 và D4.

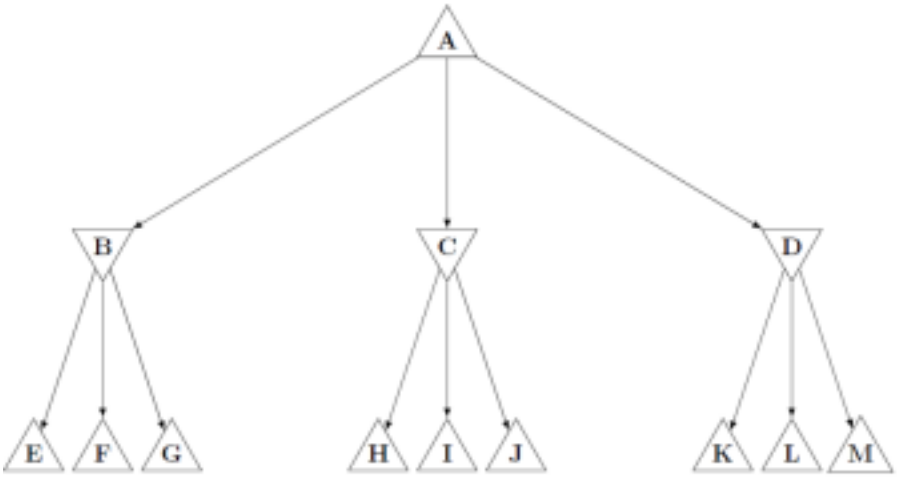
b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

Có thể di chuyển quân hậu tại C2 sang C4 để giữ nguyên một xung đột, rồi di chuyển quân hậu tại D4 sang D2 để đạt nghiệm không xung đột. Điều này cho thấy trạng thái dừng là một shoulder (vai bằng): có thể tiến tới nghiệm tốt hơn bằng cách đi ngang qua trạng thái có cùng giá trị đánh giá, mà thuật toán hiện tại không cho phép đi ngang.

- ☐ A) Không dừng
- ☐ B) Cực trị cục bộ (local optimum)
- ☐ C) Cực trị toàn cục (global optimum)
- ■ D) Vai bằng (shoulder)
- ☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

■ D → C → B

Sắp xếp từ tốt nhất đến xấu nhất:

$$19 > 14 > 4$$

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Giải:

Bước 1: Duyệt cây con D

D là nút MIN, các nút cháu có giá trị:

$$K = 23, L = 20, M = 26$$

Giá trị D:

$$D = \min(23, 20, 26) = 20$$

Do A là MAX, giá trị tốt nhất hiện tại ở gốc là:

$$\alpha_A = 20$$

Bước 2: Duyệt cây con C

C là nút MIN. Ta xét H trước:

$$H = 17$$

Tại C, giá trị $\beta_C = 17$. Vì:

$$\beta_C = 17 \leq \alpha_A = 20$$

nên các nút còn lại của C không thể làm A nhận giá trị lớn hơn 20. Vì vậy, cắt tỉa I và J.

Bước 3: Duyệt cây con B

Xét nút E đầu tiên:

$$E = 5$$

Tại B, ta có $\beta_B = 5$, trong khi $\alpha_A = 20$.

$$\beta_B = 5 \leq \alpha_A = 20$$

Do đó, cắt tỉa hai nút còn lại là F và G.

Vậy thứ tự cắt tỉa: I, J trước; sau đó F, G.

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

☐ Có

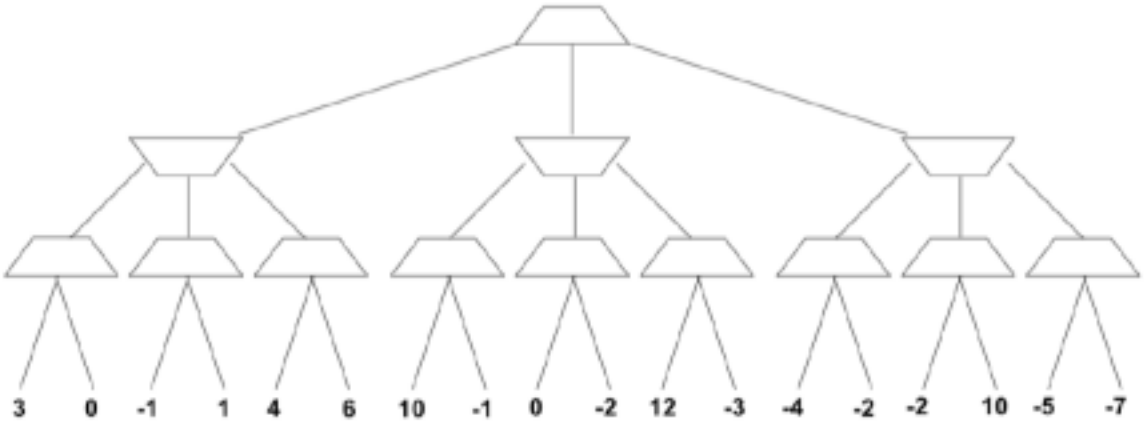
■ Không

Giải thích:

lpha-Beta Pruning chuẩn không thay đổi kết quả Minimax khi các nhánh được cắt bỏ dựa trên các cận hợp lệ và việc tìm kiếm vẫn tính giá trị chính xác của cây. Tuy nhiên, đề giới hạn độ sâu đến các nút cháu E, F, G, … và sử dụng giá trị đánh giá tại các nút này. Giá trị đánh giá ở độ sâu giới hạn không nhất thiết bằng giá trị Minimax thật của toàn bộ cây. Nên, kết quả tìm kiếm có giới hạn độ sâu có thể khác với kết quả Minimax khi duyệt toàn bộ cây

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

• Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.

• Ghi giá trị được trả về tại mỗi nút trung gian.

• Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.

• Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Giải:

Bước 1: Duyệt cây con MIN bên trái

Cây con bên trái có ba nút MAX với các cặp giá trị lá:

$$(3, 0), (-1, 1), (4, 6)$$

Nút MAX thứ nhất:

$$L_1 = \max(3, 0) = 3$$

Nút MIN bên trái cập nhật $\beta = 3$.

Nút MAX thứ hai nhận $\alpha = -\infty, \beta = 3$:

$$L_2 = \max(-1, 1) = 1$$

Nút MIN cập nhật:

$$\beta = \min(3, 1) = 1$$

Nút MAX thứ ba nhận $\alpha = -\infty, \beta = 1$. Khi duyệt lá đầu tiên có giá trị 4:

$$\alpha = \max(-\infty, 4) = 4$$

Vì $4 \geq 1$, ta cắt tỉa lá thứ hai có giá trị 6.

Nút MIN bên trái trả về:

$$L = \min(3, 1, \geq 4) = 1$$

Gốc MAX cập nhật $\alpha = 1$.

Bước 2: Duyệt cây con MIN ở giữa

Các nút MAX của cây con giữa có các cặp lá:

$$(10, -1), (0, -2), (12, -3)$$

Nút MAX thứ nhất:

$$M_1 = \max(10, -1) = 10$$

Nút MIN giữa cập nhật $\beta = 10$.

Nút MAX thứ hai nhận $\alpha = 1, \beta = 10$:

$$M_2 = \max(0, -2) = 0$$

Nút MIN cập nhật:

$$\beta = \min(10, 0) = 0$$

Do:

$$\beta = 0 \leq \alpha = 1$$

nên cắt tỉa toàn bộ nút MAX thứ ba ở giữa, bao gồm hai lá 12 và -3.

Nút MIN giữa trả về 0. Gốc MAX vẫn giữ $\alpha = 1$.

Bước 3: Duyệt cây con MIN bên phải

Nút MAX đầu tiên có hai lá:

$$R_1 = \max(-4, -2) = -2$$

Nút MIN bên phải cập nhật $\beta = -2$.

Vì:

$$\beta = -2 \leq \alpha = 1$$

ta cắt tỉa hai nút MAX còn lại. Các lá không được thăm là:

Nút MAX thứ hai: $(-2, 10)$.

Nút MAX thứ ba: $(-5, -7)$.

Nút MIN bên phải trả về -2.

Ở nút MAX trái thứ 3, giá trị 4 chỉ là giá trị tốt nhất đã quan sát khi cắt tỉa; giá trị Minimax chính xác của nút đó nếu duyệt toàn bộ là 6.

Kết quả tại gốc

Gốc MAX chọn giá trị lớn nhất trong ba cây con:

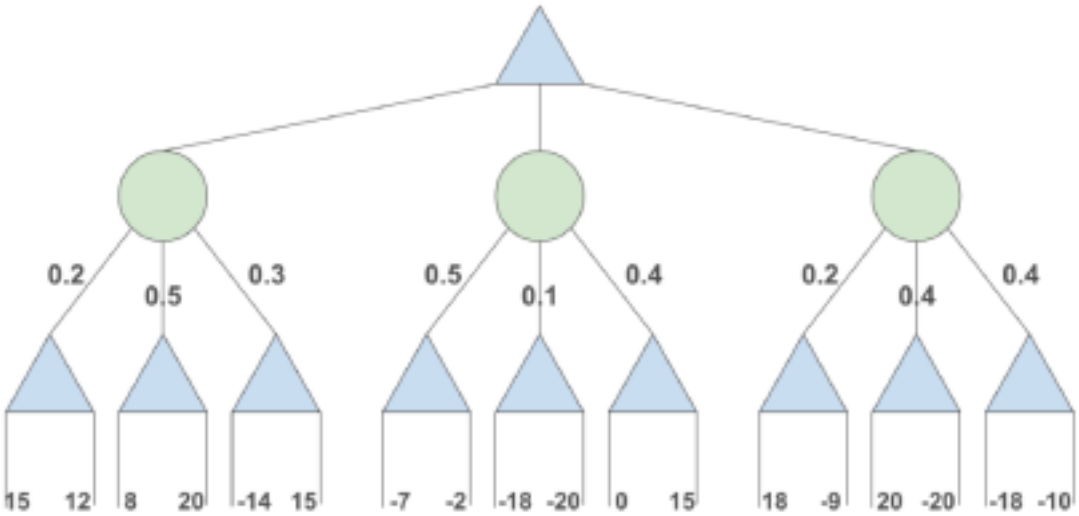
$$V_{root} = \max(1, 0, -2) = 1$$

Giá trị Minimax tại gốc là 1. Nước đi tối ưu của MAX là nhánh trái.

Các nhánh bị cắt tỉa gồm: lá 6; hai lá 12, -3; toàn bộ hai cây con có các lá (-2, 10) và (-5, -7).

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Giải:

Chance node bên trái

Ba nút MAX lần lượt có các cặp lá:

$$(15, 12), (8, 20), (-14, 15)$$

Giá trị ba nút MAX:

$$V_1 = 15, V_2 = 20, V_3 = 15$$

Với các xác suất $0.2, 0.5, 0.3$:

$$V_L = 0.2(15) + 0.5(20) + 0.3(15) = 17.5$$

Chance node ở giữa

Các cặp lá:

$$(-7, -2), (-18, -20), (0, 15)$$

Các giá trị MAX:

$$V_1 = -2, V_2 = -18, V_3 = 15$$

Với các xác suất $0.5, 0.1, 0.4$:

$V_M = 0.5(-2) + 0.1(-18) + 0.4(15) = 3.2$

Chance node bên phải

Các cặp lá:

$$(18, -9), (20, -20), (-18, -10)$$

Các giá trị MAX:

$$V_1 = 18, V_2 = 20, V_3 = -10$$

Với các xác suất $0.2, 0.4, 0.4$:

$V_R = 0.2(18) + 0.4(20) + 0.4(-10) = 7.6$

Giá trị tại gốc MAX

$V_{root} = 17.5$

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

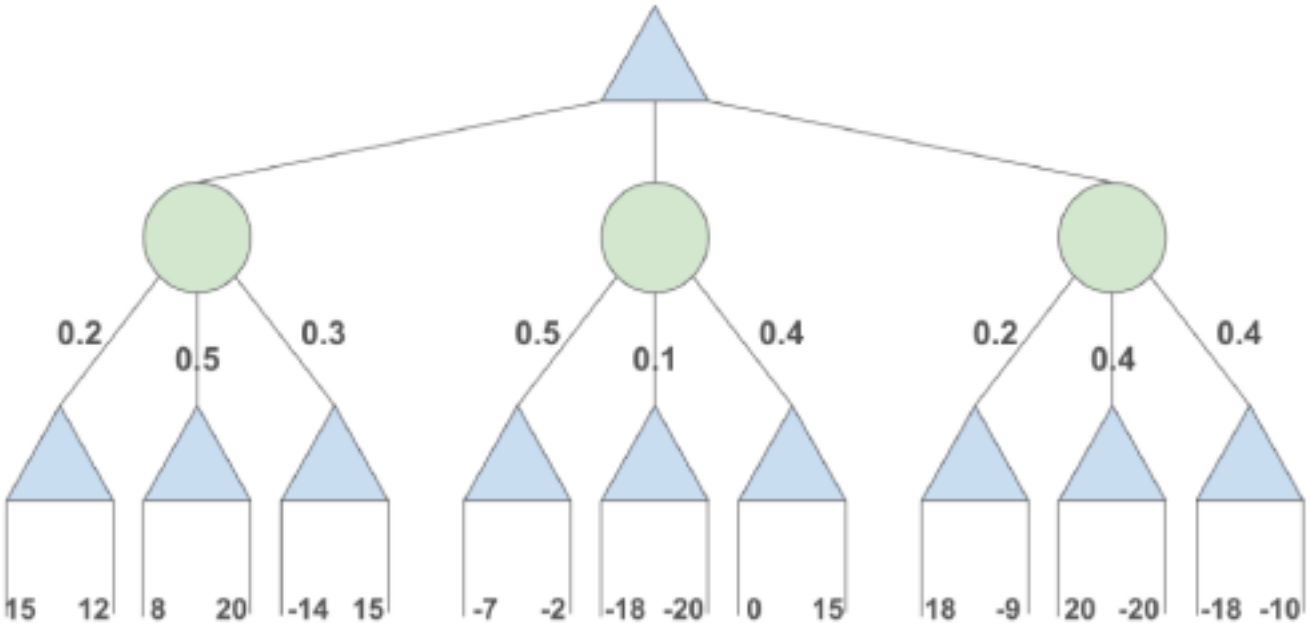
Giải:

Không có nhánh nào có thể cắt tỉa một cách an toàn chỉ dựa trên các giá trị đã quan sát, do:

- Tại nút MAX, một lá chưa duyệt có thể có giá trị cao hơn lá đã duyệt.
- Tại chance node, một nhánh chưa duyệt có thể đóng góp giá trị kỳ vọng rất lớn hoặc rất nhỏ vì ta không có cận trên hay cận dưới của giá trị lá.

- Vì thế, chưa thể chứng minh phần chưa duyệt không làm thay đổi giá trị kỳ vọng hoặc quyết định tối ưu tại gốc.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Giải:

Bước 1: Tính chance node bên trái

Ta tính chính xác được:

$$V_L = 17.5$$

Đây là giá trị tốt nhất tại gốc MAX hiện tại.

Bước 2: Xét chance node ở giữa

Nút MAX đầu tiên có hai lá -7 và -2, nên trả về:

$$V_1 = \max(-7, -2) = -2$$

Đóng góp hiện tại vào kỳ vọng là:

$$0.5(-2) = -1$$

Tổng xác suất của hai nhánh còn lại là:

$$0.1 + 0.4 = 0.5$$

Vì mỗi giá trị lá không vượt quá 20, giá trị kỳ vọng lớn nhất mà chance node giữa có thể đạt là:

$$V_M^{\max} = -1 + 0.5(20) = 9$$

Mà:

$$9 < 17.5$$

Vì vậy, dù hai nút MAX còn lại có giá trị cao đến mức nào trong giới hạn cho phép, chance node giữa cũng không thể vượt chance node bên trái.

Cắt tỉa hai nút MAX còn lại của chance node giữa, gồm các lá (-18, -20) và (0, 15).

Bước 3: Xét chance node bên phải

MAX đầu tiên có hai lá 18 và -9:

$$V_1 = 18$$

Cận trên của chance node bên phải lúc này là:

$$0.2(18) + 0.8(20) = 19.6$$

Vì $19.6 > 17.5$, chưa thể cắt tỉa cả phần còn lại của chance node bên phải.

Tiếp theo, nút MAX thứ hai có cặp lá (20, -20). Ta duyệt lá đầu tiên có giá trị 20. Vì mọi lá không vượt quá 20, ta biết ngay:

$$\max(20, -20) = 20$$

Do đó, có thể cắt tỉa lá -20 mà không cần thăm nó.

Cận trên của chance node phải vẫn là:

$$0.2(18) + 0.4(20) + 0.4(20) = 19.6$$

Cận này vẫn lớn hơn 17.5, nên cần đánh giá nút MAX cuối cùng:

$$V_3 = \max(-18, -10) = -10$$

Giá trị chính xác của chance node phải là:

$$V_R = 0.2(18) + 0.4(20) + 0.4(-10) = 7.6$$

Các phần bị cắt tỉa:

- Hai nút MAX còn lại của chance node ở giữa, cùng toàn bộ các lá của chúng.
- Lá -20 thứ hai của nút MAX có cặp lá (20, -20) ở chance node bên phải.

Gốc MAX vẫn chọn chance node bên trái với giá trị:

$V_{root} = 17.5$

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Tiêu chí	Forward Checking	AC-3
Thời điểm thực hiện	Sau khi gán một biến	Khi khởi tạo hoặc khi cần duy trì tính nhất quán cung
Phạm vi lan truyền	Xóa các giá trị không phù hợp ở biến chưa gán có liên quan trực tiếp đến biến vừa gán	Kiểm tra và sửa các cung trên đồ thị ràng buộc, sau đó tiếp tục lan truyền khi miền thay đổi
Chi phí tính toán	Thường thấp hơn	Thường cao hơn do phải xử lý nhiều cung và có thể lặp lại
Phát hiện mâu thuẫn	Có thể phát hiện miền của biến kế cận rỗng sau một phép gán	Có thể phát hiện mâu thuẫn sớm hơn nhờ lan truyền qua nhiều cung
Ưu điểm	Đơn giản, nhanh, phù hợp kết hợp Backtracking	Thu hẹp miền mạnh hơn và loại được nhiều giá trị không thể tham gia nghiệm

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

MRV chọn biến chưa gán có ít giá trị hợp lệ nhất trong miền hiện tại.

Lý do: biến có ít lựa chọn thường khó thỏa mãn nhất. Giải quyết biến này trước giúp phát hiện mâu thuẫn sớm, tránh dành nhiều thời gian cho nhánh chắc chắn không có nghiệm.

ii) Least Constraining Value (LCV)

LCV chọn giá trị gây ít hạn chế nhất cho các biến chưa gán có liên quan.

Lý do: giá trị đó giữ lại nhiều lựa chọn hợp lệ cho các biến khác, tăng khả năng hoàn thành nghiệm mà không phải

quay lui.

.. Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

• Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.

• Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.

• Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.

• Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C. • Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng. a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

.....

.....

.....

.....

Bổ sung hai ràng buộc sau:

• A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.

• D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

.....

.....

.....

.....

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

.....

.....

.....

.....

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4

B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

.....

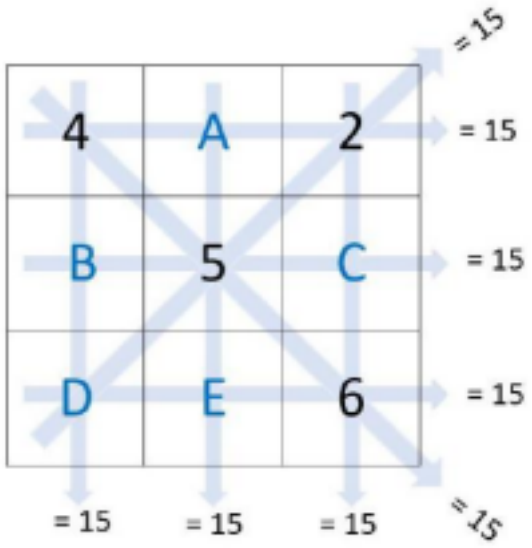
.....

.....

.....

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

.....

.....

.....

.....

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.

.....

.....

.....

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

.....

.....

.....

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}

• Sunny = {Đỏ, Vàng, Hồng}

• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

.....

.....

.....

.....

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

.....

.....

.....

.....

--- Hết ---